

A CULTURAL AND OPERATIONAL TRANSFORMATION IN CLINICAL DATA MANAGEMENT



Project Introduction



PRIMECARE THERAPY CENTER



Project Focus & Development

- Designed and implemented a complete relational database system for PrimeCare Therapy Center
- Built using structured SQL techniques following industry best practices
- Covers database creation, data insertion, complex queries, and transaction management

Project Goals & Benefits

- Supports daily clinic operations end-to-end
- Manages patients, therapists, appointments, services, payments, and clinical records
- Creates an organized, reliable, and efficient data environment
- Improves data management and supports evidence-based business decision-making



Foundation Secured. Project Ready for Deployment.

Database Creation — DDL Implementation



Core DDL Steps

- Created the **PrimeCareDB** database from scratch
- Schema designed to conform to **Third Normal Form (3NF)**
- Implemented both main (parent) and child tables
- Added **Primary Keys** and **Foreign Keys** to enforce relational integrity

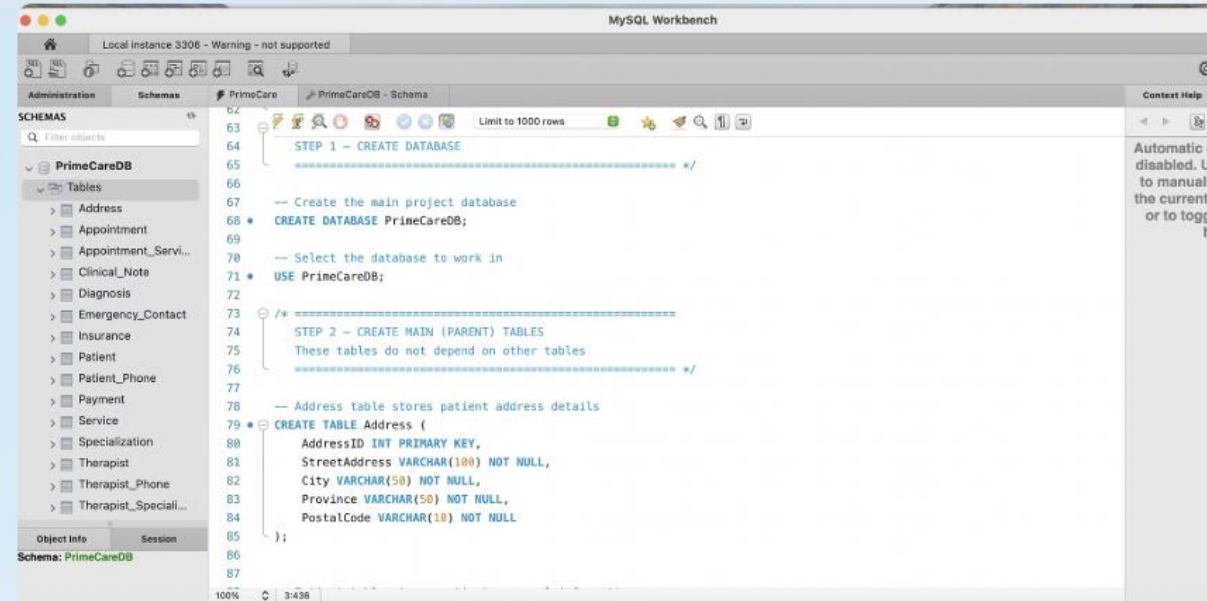


Integrity & Order



- Applied constraints to maintain data integrity at every step
- Ensured correct table creation order to satisfy dependency requirements

 Structural Foundation Secured

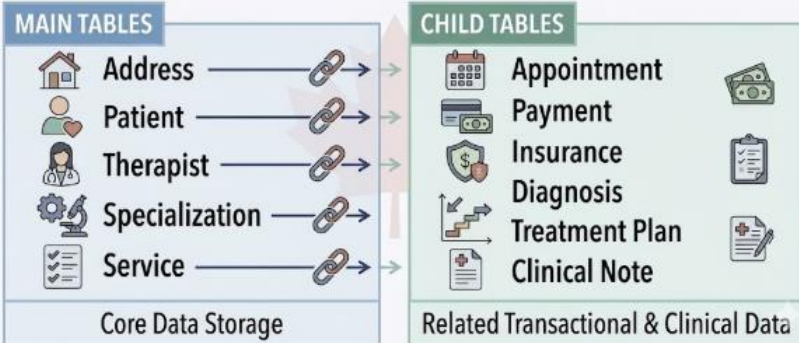


Main Tables & Child Tables

PRIMECAREDB SCHEMA

Main Tables and Child Tables

PrimeCareDB



Main Tables — Core Data Storage

- **Address** — physical location records
- **Patient** — demographic and contact data
- **Therapist** — staff credentials and profiles
- **Specialization** — therapy disciplines offered
- **Service** — catalog of clinical services

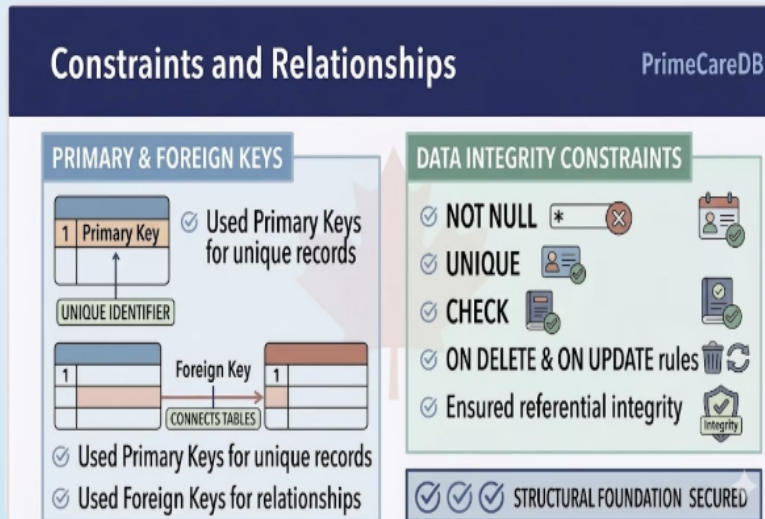
Child Tables — Transactional & Clinical Data

- Appointment · Payment · Insurance
- Diagnosis · Treatment Plan · Clinical Note

Constraints & Relationships

Primary & Foreign Keys

- **Primary Keys** guarantee a unique identifier for every record
- **Foreign Keys** link child tables back to their parent, enforcing referential integrity



Data Integrity Constraints

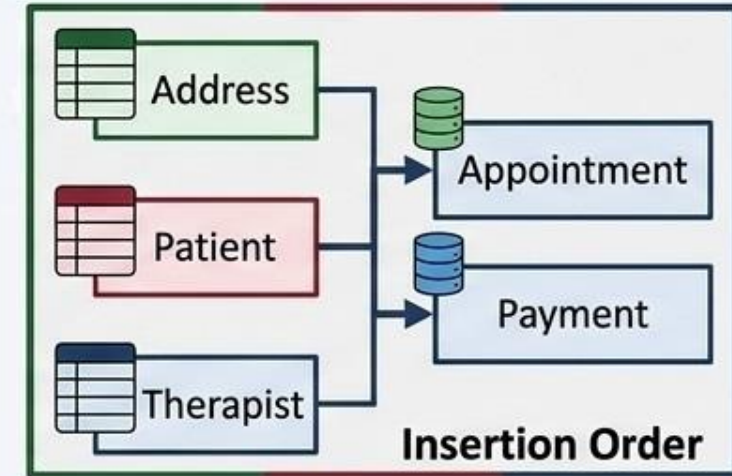
- **NOT NULL** — required fields cannot be left empty
- **UNIQUE** — prevents duplicate values in key columns
- **CHECK** — validates data against business rules
- **ON DELETE / ON UPDATE** cascade rules protect related records

Data Insertion (DML)

PrimeCare
Therapy Center

DML Insertion Summary

- ✓ Inserted sample data into all tables of the database
- ✓ Ensured at least 5 records per table as required
- ✓ Followed correct insertion order:
 - Parent tables first (Address, Patient, Therapist)
 - Then child tables (Appointment, Payment, etc.)
- ✓ Maintained all primary and foreign key relationships
- ✓ Used realistic data based on PrimeCare Therapy Center scenario



```
INSERT INTO Address VALUES
(1, '123 Main St', 'Toronto', 'ON', 'M1A1A1'),
(2, '456 Oak Ave', 'Montreal', 'QC', 'H1A1A1'),
(3, '789 Pine Rd', 'Vancouver', 'BC', 'V1A1A1'),
(4, '321 Maple St', 'Calgary', 'AB', 'T1A1A1'),
(5, '654 Cedar Ln', 'Ottawa', 'ON', 'K1A1A1');
```


Validation & Importance

DATA VERIFICATION

```
SELECT * FROM Patient;
```

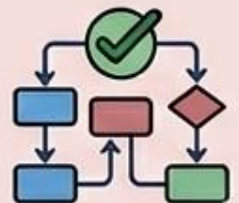


Result Grid						
Filter Rows:		Edit:		Export/Import:		
PatientID	FirstName	LastName	DateOfBirth	Email	AddressID	
1001	Sarah	Lee	1995-06-12	sarah@email.com	1	
1002	Marcus	Brown	1990-03-22	marcus@email.com	2	
1003	Aisha	Khan	1998-11-05	aisha@email.com	3	
1004	John	Smith	1985-01-15	john@email.com	4	
1005	Emily	Davis	2000-07-30	emily@email.com	5	

- ✓ Ensured no constraint violations
- ✓ Confirmed no duplicate or missing data

IMPORTANCE OF THIS STEP

- ✓ Allows testing of complex queries
- ✓ Ensures database is functional and reliable
- ✓ Data reflects real-world operations of a therapy center



Data Manipulation

UPDATE · DELETE · INSERT — PrimeCare Therapy Center

UPDATE

5 Statements

Raise active therapist rates by +10%

Therapist Sophia: Inactive → Active

Appt. 3002 status: Scheduled → Completed

Update patient Sarah Lee's email

Adjust BlueCross coverage to 95%

DELETE

3 Statements

Remove \$0 payment (cancelled appt.)

Remove Home phone — Patient 1001

Remove Therapist 203 specialization

INSERT

7 New Records

New address + Patient: Liam Nguyen (ID 1006)

Mobile phone + Emergency contact

Insurance: BlueCross 80%

Appointment 3006 — Apr 10, 2026

Service link + Clinical Note (first session)

Transaction Management

START TRANSACTION · COMMIT · ROLLBACK — Ensuring Data Integrity

Transaction 1

STEPS EXECUTED:

- 1 INSERT new appointment (ID: 3007)
- 2 LINK service to appointment
- 3 RECORD payment — \$110.00 (Debit)
- 4 ADD clinical note for John Smith

✓ COMMITTED

Transaction 2

STEPS EXECUTED:

- 1 INSERT appointment (ID: 3008)
- 2 LINK service to appointment
- 3 INVALID payment: -\$50.00
- 4 Violates CHECK (Amount ≥ 0) ← ERROR

✗ ROLLED BACK

Transaction 3

STEPS EXECUTED:

- 1 ASSIGN specialization to Therapist 203
- 2 INSERT diagnosis for Patient 1006
- 3 ADD treatment plan — bi-weekly sessions

✓ COMMITTED

Summary & Key Takeaways

Data Manipulation & Transaction Management — PrimeCare Therapy Center

5

UPDATE
Statements

3

DELETE
Statements

7

Records
Inserted

3

Transactions
Managed

KEY TAKEAWAYS

UPDATE and DELETE use WHERE clauses to target specific records — protecting unrelated data from accidental changes.

COMMIT permanently saves all changes in a transaction — only issued when every step succeeds.

ROLLBACK undoes an entire transaction — used when any step fails, keeping the database consistent.

Transaction 2 demonstrated a CHECK constraint violation triggering a full rollback of all operations.

Basic Data Retrieval

```
SELECT
  PatientID,
  FirstName,
  LastName,
  Email
FROM patient;
```



	PatientID	FirstName	LastName	Email
▶	1001	Sarah	Lee	sarah.lee.updated@email.com
	1002	Marcus	Brown	marcus@email.com
	1003	Aisha	Khan	aisha@email.com
	1004	John	Smith	john@email.com
	1005	Emily	Davis	emily@email.com
	1006	Liam	Nguyen	liam@email.com
*	NULL	NULL	NULL	NULL

```
SELECT
  TherapistID,
  FirstName,
  LastName,
  HourlyRate
FROM therapist;
```



	TherapistID	FirstName	LastName	HourlyRate
▶	201	Alice	Johnson	88.00
	202	David	Miller	99.00
	203	Sophia	Wilson	85.00
	204	James	Taylor	82.50
	205	Olivia	Anderson	104.50
*	NULL	NULL	NULL	NULL

- Retrieved data using SELECT statements
- Verified patient and therapist records
- Ensured data is accurate before performing joins.

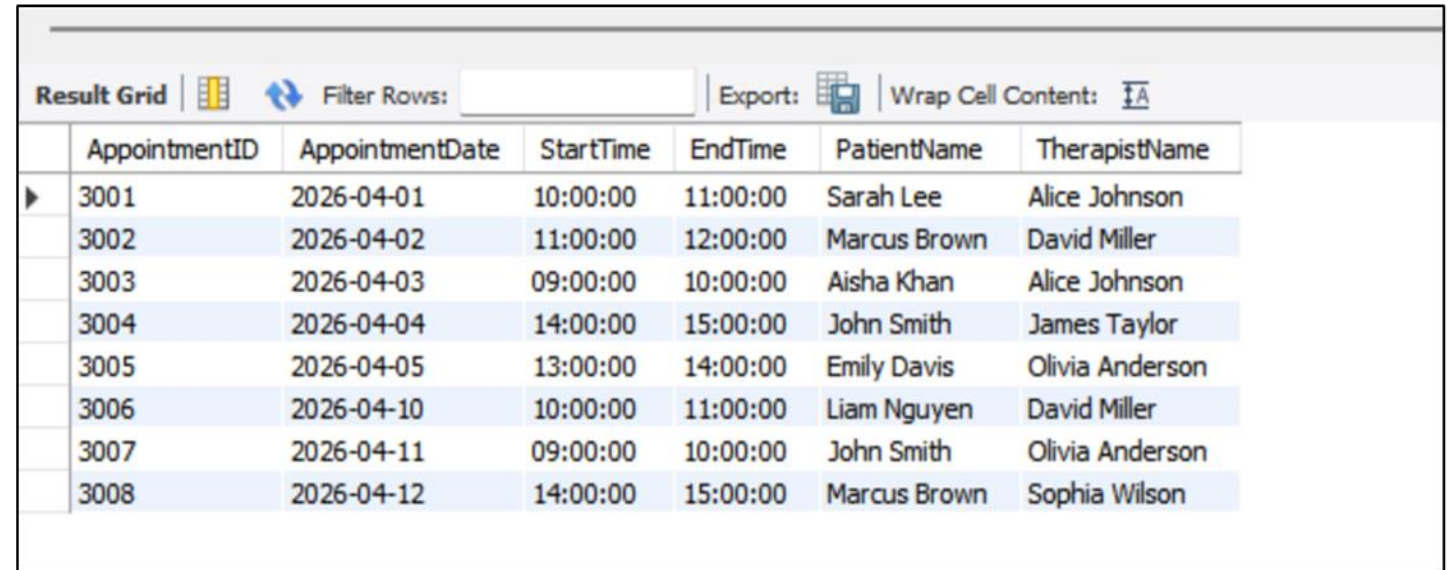
Combining Data With Inner Join

```
SELECT
  a.AppointmentID,
  a.AppointmentDate,
  a.StartTime,
  a.EndTime,

  CONCAT(p.FirstName, ' ', p.LastName) AS PatientName,
  CONCAT(t.FirstName, ' ', t.LastName) AS TherapistName

FROM appointment a
INNER JOIN patient p
  ON a.PatientID = p.PatientID
INNER JOIN therapist t
  ON a.TherapistID = t.TherapistID

ORDER BY a.appointmentDate;
```



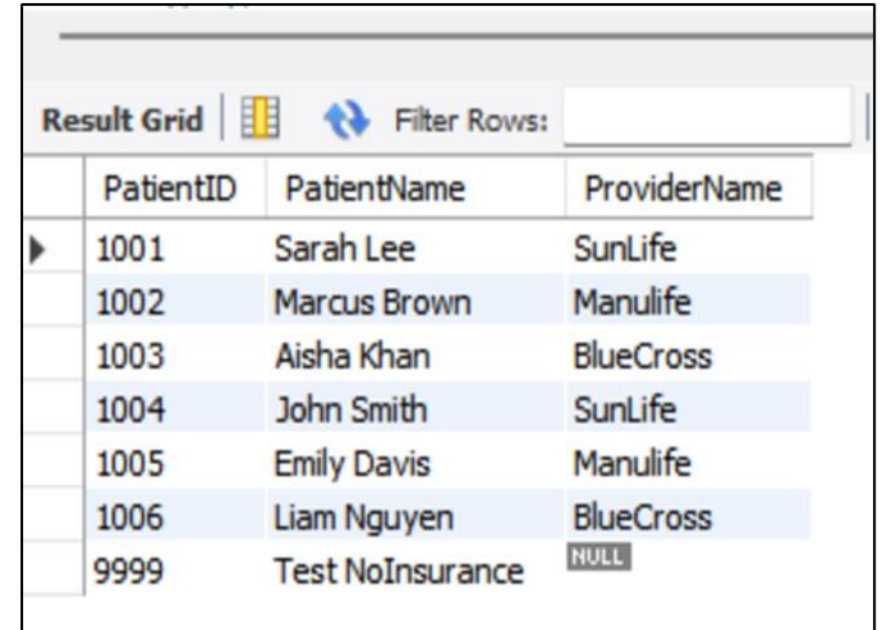
	AppointmentID	AppointmentDate	StartTime	EndTime	PatientName	TherapistName
▶	3001	2026-04-01	10:00:00	11:00:00	Sarah Lee	Alice Johnson
	3002	2026-04-02	11:00:00	12:00:00	Marcus Brown	David Miller
	3003	2026-04-03	09:00:00	10:00:00	Aisha Khan	Alice Johnson
	3004	2026-04-04	14:00:00	15:00:00	John Smith	James Taylor
	3005	2026-04-05	13:00:00	14:00:00	Emily Davis	Olivia Anderson
	3006	2026-04-10	10:00:00	11:00:00	Liam Nguyen	David Miller
	3007	2026-04-11	09:00:00	10:00:00	John Smith	Olivia Anderson
	3008	2026-04-12	14:00:00	15:00:00	Marcus Brown	Sophia Wilson

- Combined appointment, patient, and therapist tables
- Replaced IDs with readable names using CONCAT
- Shows only valid records with matching data
- Sorted by appointment date for clarity

Handling Missing Data With Left Join

```
SELECT
  p.PatientID,
  CONCAT (p.FirstName, ' ', p.LastName) AS PatientName,
  i.ProviderName

FROM patient p
LEFT JOIN insurance i
  ON p.PatientID = i.PatientID;
```



	PatientID	PatientName	ProviderName
▶	1001	Sarah Lee	SunLife
	1002	Marcus Brown	Manulife
	1003	Aisha Khan	BlueCross
	1004	John Smith	SunLife
	1005	Emily Davis	Manulife
	1006	Liam Nguyen	BlueCross
	9999	Test NoInsurance	NULL

- Retrieved all patients, including those without insurance
- Demonstrated NULL values using a test patient
- Shows unmatched records instead of excluding them
- Helps identify missing insurance information

Multi-table Join

```
SELECT
  a.AppointmentID,
  a.AppointmentDate,
  a.StartTime,
  a.EndTime,

  CONCAT(p.FirstName, ' ', p.LastName) AS PatientName,
  CONCAT(t.FirstName, ' ', t.LastName) AS TherapistName,

  s.ServiceName

FROM appointment a
INNER JOIN patient p
  ON a.PatientID = p.PatientID
INNER JOIN therapist t
  ON a.TherapistID = t.TherapistID
INNER JOIN appointment_service aps
  ON a.AppointmentID = aps.AppointmentID
INNER JOIN service s
  ON aps.ServiceID = s.ServiceID

ORDER BY a.AppointmentDate;
```

	AppointmentID	AppointmentDate	StartTime	EndTime	PatientName	TherapistName	ServiceName
▶	3001	2026-04-01	10:00:00	11:00:00	Sarah Lee	Alice Johnson	Individual Therapy
	3001	2026-04-01	10:00:00	11:00:00	Sarah Lee	Alice Johnson	Assessment
	3002	2026-04-02	11:00:00	12:00:00	Marcus Brown	David Miller	Family Therapy
	3003	2026-04-03	09:00:00	10:00:00	Aisha Khan	Alice Johnson	Group Therapy
	3005	2026-04-05	13:00:00	14:00:00	Emily Davis	Olivia Anderson	Rehabilitation Session
	3006	2026-04-10	10:00:00	11:00:00	Liam Nguyen	David Miller	Individual Therapy
	3007	2026-04-11	09:00:00	10:00:00	John Smith	Olivia Anderson	Rehabilitation Session
	3008	2026-04-12	14:00:00	15:00:00	Marcus Brown	Sophia Wilson	Family Therapy

- Combined appointment, patient, therapist, and service tables
- Used a junction table to link services to appointments
- One appointment can have multiple services
- Provides a complete view of each appointment

Advanced SQL Queries & Reporting

GROUP BY · COUNT · SUM · AVG

Subqueries · Business Intelligence

```
SELECT COUNT(*) AS TotalAppointments
```

```
FROM Therapist t
```

```
LEFT JOIN Appointment a
```

```
    ON t.TherapistID = a.TherapistID
```

```
GROUP BY t.TherapistID;
```

```
SELECT SUM(Amount) AS TotalRevenue
```

```
FROM Payment
```

```
GROUP BY PaymentMethod;
```

```
HAVING SUM(Amount) > (
```

```
    SELECT AVG(Amount) FROM Payment
```

```
);
```

GROUP BY Queries — Workload & Revenue

Q1 — Appointments Per Therapist

```
SELECT
  CONCAT(t.FirstName, ' ', t.LastName) AS TherapistName,
  COUNT(a.AppointmentID) AS TotalAppointments
FROM Therapist t
LEFT JOIN Appointment a ON t.TherapistID = a.TherapistID
GROUP BY t.TherapistID
ORDER BY TotalAppointments DESC;
```

TherapistName	TotalAppointments
Alice Johnson	2
David Miller	1
Olivia Anderson	1
James Taylor	1

💡 Alice with 2 appointments, David & Olivia lead with 1 appointments each. Sophia has none flagged as inactive initially.

Q2 — Total Revenue Per Therapist

```
SELECT
  CONCAT(t.FirstName, ' ', t.LastName) AS TherapistName,
  SUM(py.Amount) AS TotalRevenue
FROM Therapist t
JOIN Appointment a ON t.TherapistID = a.TherapistID
JOIN Payment py ON a.AppointmentID = py.AppointmentID
GROUP BY t.TherapistID ORDER BY TotalRevenue DESC;
```

TherapistName	TotalRevenue (\$)
Alice Johnson	330.00
Olivia Anderson	110.00
David Miller	120.00
James Taylor	0.00

💡 Alice Johnson generates the highest revenue (\$330). James Taylor's \$0 reflects the cancelled appointment payment that was deleted.

Aggregate Functions — Payments & Appointment Status

Q3 — Avg Payment by Method

```
SELECT
  PaymentMethod,
  COUNT(*) AS NumberOfPayments,
  ROUND(AVG(Amount),2) AS AverageAmount,
  SUM(Amount) AS TotalAmount
FROM Payment
GROUP BY PaymentMethod ORDER BY TotalAmount DESC;
```

PaymentMethod	# Payments	Avg (\$)	Total (\$)
Credit Card	2	180.00	360.00
Debit	1	80.00	80.00
Cash	2	60.00	120.00

💡 Credit Card is the most preferred method, generating the highest total revenue (\$360).

Q4 — Appointment Status Summary

```
SELECT
  Status,
  COUNT(*) AS NumberOfAppointments
FROM Appointment
GROUP BY Status
ORDER BY NumberOfAppointments DESC;
```

Status	NumberOfAppointments
Completed	3
Cancelled	1
Scheduled	1

💡 71% of appointments are Completed, strong clinic performance. 1 Cancelled (Appt 3004). Appointment 3008 was rolled back and does not appear.

3

Completed

1

Cancelled

1

Scheduled

GROUP BY Queries — Workload & Revenue

Q5 — Total Revenue Per Month

```
SELECT
  YEAR(PaymentDate) AS PaymentYear,
  MONTH(PaymentDate) AS PaymentMonth,
  COUNT(*)          AS NumberOfPayments,
  SUM(Amount)        AS MonthlyRevenue
FROM Payment
GROUP BY YEAR(PaymentDate), MONTH(PaymentDate)
```

Payment Year	Monthly Revenue (\$)
2026	560

💡 \$560 earned monthly, this enables revenue trend analysis over time.

Q6 — Services Most Frequently Used

```
SELECT
  s.ServiceID,
  s.ServiceName,
  s.CPTCode,
  COUNT(aps.AppointmentID) AS TimesUsed,
  SUM(aps.Quantity)        AS TotalQuantity,
  SUM(aps.AppliedRate)     AS TotalBilled
FROM Service s
LEFT JOIN Appointment_Service aps
  ON s.ServiceID = aps.ServiceID
GROUP BY s.ServiceID, s.ServiceName, s.CPTCode
ORDER BY TimesUsed DESC;
```

ServiceName	Total Billed
Individual Therapy	100.00
Family Therapy	120.00
Group Therapy	80.00
Assessment	150.00
Rehabilitation Session	110.00

💡 Identifies which services drive the most demand. Supports inventory, staffing, and marketing decisions.

Subqueries — Patient Value & Therapist Compensation

Q7 — Patients Who Paid Above Average

```
SELECT
CONCAT(p.FirstName, ' ', p.LastName) AS PatientName,
SUM(py.Amount) AS TotalPaid
FROM Patient p
JOIN Appointment a ON p.PatientID = a.PatientID
JOIN Payment py ON a.AppointmentID = py.AppointmentID
GROUP BY p.PatientID
HAVING SUM(py.Amount) > (
    SELECT AVG(Amount) FROM Payment
)
```

PatientName	TotalPaid (\$)
Sarah Lee	250.00
Marcus Brown	120.00

💡 Avg payment = \$134.00. Only Sarah Lee (\$250) exceeds this she received both Individual Therapy and Assessment in one appointment.

Subquery: (SELECT AVG(Amount) FROM Payment) → returns \$134.00 used in HAVING clause

Q8 — Therapists with Above-Average Rate

```
SELECT
CONCAT(FirstName, ' ', LastName) AS TherapistName,
HourlyRate,
TherapistStatus
FROM Therapist
WHERE HourlyRate > (
    SELECT AVG(HourlyRate) FROM Therapist
)
```

TherapistName	HourlyRate (\$)	Status
Olivia Anderson	95.00	Active
David Miller	90.00	Active

💡 Avg hourly rate = \$91.80 (after 10% raise for active therapists). Olivia (\$95.00) and David (\$90.00) are the highest earners.

Subquery: (SELECT AVG(HourlyRate) FROM Therapist) → returns \$91.80 used in WHERE clause

Q9 — Insurance Coverage by Provider

```
SELECT ProviderName,
       COUNT(PatientID) AS NumberOfPatients,
       ROUND(AVG(CoveragePercent),2) AS AvgCoverage,
       MIN(CoveragePercent) AS MinCoverage,
       MAX(CoveragePercent) AS MaxCoverage
FROM Insurance
GROUP BY ProviderName ORDER BY NumberOfPatients DESC;
```

ProviderName	Patients	Avg%	Min%	Max%
BlueCross	1	90.00	90.00	90.00
SunLife	2	77.50	75.00	80.00
Manulife	2	77.50	70.00	85.00

💡 BlueCross provides the highest average coverage (90%). Manulife and Sunlife providers serve 2 patients each. Useful for billing and claims forecasting.

Q10 — Executive Dashboard (Business Summary)

```
SELECT
  (SELECT COUNT(*) FROM Patient) AS TotalPatients,
  (SELECT COUNT(*) FROM Therapist
   WHERE TherapistStatus='Active') AS ActiveTherapists,
  (SELECT COUNT(*) FROM Appointment) AS TotalAppointments,
  (SELECT COUNT(*) FROM Appointment
   WHERE Status='Completed') AS CompletedAppts,
  (SELECT SUM(Amount) FROM Payment) AS TotalRevenue,
  (SELECT ROUND(AVG(Amount),2)
   FROM Payment) AS AvgPayment;
```

5

Total Patients

4

Active Therapists

5

Total Appointments

3

Completed

\$560

Total Revenue

\$112

Avg Payment

Advanced SQL Queries

Summary & Key Takeaways — PrimeCare Therapy Center

10

Queries Written

4

GROUP BY Queries

4

Aggregate Functions

2

Subqueries

KEY TAKEAWAYS

GROUP BY with aggregate functions (COUNT, SUM, AVG) transforms raw data into actionable business intelligence.

Subqueries allow dynamic comparisons e.g., identifying patients paying above the current average without hardcoding values.

The Executive Dashboard (Q10) uses 6 nested subqueries to deliver a full clinic performance snapshot in a single result row.

HAVING filters grouped results unlike WHERE which filters individual rows critical for revenue and threshold analysis.

THANK YOU



PrimeCare⁺
Therapy Center